

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1714

COURSE OUTCOME COVERED

CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.

(BTL 5)

Assessment Tool Weightage: 20%

Duration: 1 hour

QUESTIONS: 5

Total Marks:50

Annexure A

Reverse Engineering

Code of Conduct:

I **Vempati Murale Karrtik (192312567)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Vempati Murale Karrtik

Signature of the student:

1. Aim

The aim of this design task is to reverse engineer a realistic high-level social media content moderation system that can automatically identify and act on harmful content within seconds of posting. The proposed design explains how text and image content can be ingested, transformed using PySpark RDD operations, analyzed by AI models, and evaluated by collaborating intelligent agents before an automated action or escalation to a human reviewer.

2. Understanding of the Industry Problem

Social media platforms receive a continuous stream of user-generated posts containing text, images, captions, comments and metadata. The moderation system must process this content rapidly while handling very large traffic volumes. It must distinguish harmless content from potentially harmful content, minimize false positives, protect users, preserve evidence for review, and provide consistent enforcement of platform policies.

Key Industry Constraints

- Very high and continuously changing content volume.
- Low latency because harmful content may spread within seconds.
- Multiple modalities including text, images and associated metadata.
- Large variation in language, slang, spelling, context and cultural references.
- Need to detect both known harmful patterns and previously unseen behavior.
- False positives can unfairly restrict legitimate users; false negatives can expose users to harmful material.
- Sensitive moderation decisions require auditability, access control and privacy-aware data handling.
- Borderline cases should be escalated to trained human reviewers rather than relying only on automation.

Expected Outcome

The reverse-engineered system should transform a newly submitted post into a moderation decision through the following logical flow:

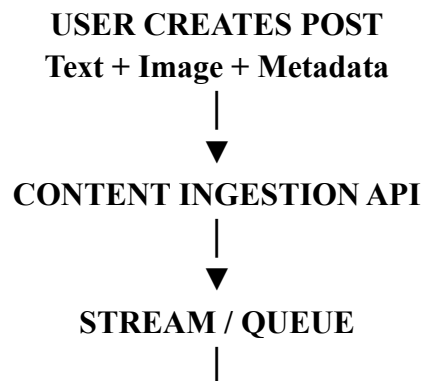
**POST → INGESTION → RDD PROCESSING → TEXT/IMAGE FEATURES → AI
ANALYSIS → AGENT COLLABORATION → RISK SCORE → ACTION OR HUMAN
ESCALATION**

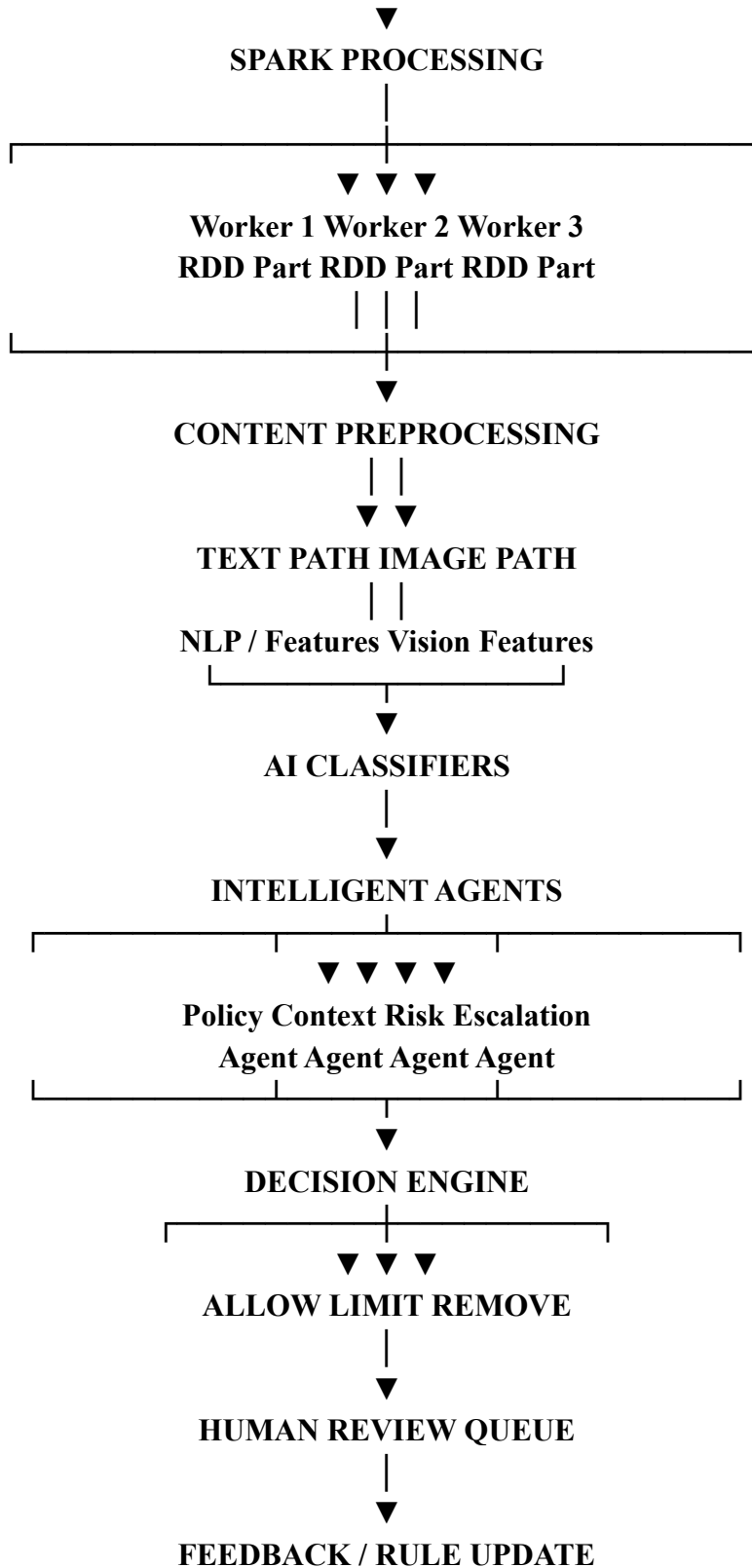
3. Functional Requirements

Requirement	Description
Content ingestion	Receive text, images and post metadata.
Distributed processing	Process large content volumes using Spark RDD partitions.
Text analysis	Extract language, keywords, semantic and contextual features.
Image analysis	Extract visual features and detect potentially harmful visual content.
Risk scoring	Combine model outputs into a moderation risk estimate.
Automated action	Allow high-confidence policy decisions to be acted upon.
Human escalation	Send uncertain or high-impact cases to reviewers.
Continuous improvement	Use reviewed outcomes to improve models and moderation rules.

4. Reverse-Engineered System Architecture – 25%

The following architecture is a conceptual reverse engineering of how a large-scale moderation platform could be organized. It is not claimed to reproduce any particular company's proprietary implementation. The design uses industry-standard distributed processing and human-in-the-loop moderation concepts.





5. Major System Components

Component	Purpose
Content Ingestion	Accepts posts and metadata immediately after submission.
Stream/Queue	Buffers content and distributes work to processing nodes.
Spark Cluster	Runs distributed transformations and analysis over partitions.
Text Processing	Normalizes text and extracts linguistic/semantic features.
Image Processing	Extracts visual features and detects potentially harmful visual patterns.
AI Models	Produce modality-specific classification scores.
Policy Agent	Checks outputs against moderation policies.
Context Agent	Uses surrounding context and user/report information where permitted.
Risk Agent	Combines evidence into an overall risk score.
Escalation Agent	Routes uncertain/high-impact cases to human reviewers.
Decision Engine	Applies thresholds and selects the moderation action.
Feedback Store	Stores validated decisions for future model/rule improvement.

Reverse Engineering a Social Media Content Moderation System

6. Application of Engineering Concepts

6.1 Text Processing

Text content is first normalized before analysis. Typical preprocessing can include Unicode normalization, language identification, tokenization, removal or masking of irrelevant artifacts, and extraction of contextual features. Modern NLP models can then produce semantic representations for classification.

```
text_rdd = posts.map(lambda p: (p["post_id"], normalize_text(p["text"])))
```

```
language_rdd = text_rdd.mapValues(detect_language)
```

```
features_rdd = language_rdd.mapValues(extract_text_features)
```

6.2 Image Processing

Images can be decoded and passed to a computer-vision model that produces features or category probabilities. In a scalable architecture, image preprocessing and model inference can be distributed across worker processes or dedicated inference services.

Example image features include object categories, scene characteristics, text detected inside the image, visual similarity indicators and model-generated safety scores.

6.3 RDD Transformations

RDD Operation	Moderation Use
map()	Convert raw records into structured content objects.
filter()	Remove invalid records or isolate records requiring analysis.
mapValues()	Apply feature extraction while preserving content IDs.
flatMap()	Generate multiple tokens, tags or indicators from one record.
reduceByKey()	Aggregate reports, events or risk evidence by post/user/content ID.
join()	Combine content features with approved metadata or policy information.
distinct()	Identify unique indicators or repeated patterns.
cache()	Keep frequently reused intermediate data available for later stages.

6.4 Example Distributed Processing

Suppose one million posts arrive during a short interval. Instead of processing all records on a single machine, Spark partitions the RDD across worker nodes. Each worker performs transformations on its assigned partition, and aggregation operations combine the results.

**1,000,000 POSTS → RDD PARTITIONS → WORKER NODES → TEXT/IMAGE
FEATURES → MODEL SCORES → AGGREGATED MODERATION RESULTS**

7. Example Input Records

Post ID	Text / Content	Image	Initial Observation
P001	Have a great day everyone!	No	Ordinary text
P002	[repeated abusive phrase]	No	Potential harassment
P003	[caption describing an unsafe act]	Yes	Requires multimodal analysis
P004	New product launch today	Yes	Likely benign
P005	[threat-like statement]	Yes	High-priority review candidate

The harmful examples are intentionally represented descriptively rather than reproducing harmful content. They demonstrate the data-processing design without providing unsafe material.

8. Intelligent Agent Integration – 15%

Intelligent agents are used as specialized decision-support components. Each agent receives relevant evidence, performs a focused task and passes structured results to the next agent. The final moderation decision is based on combined evidence rather than a single score.

Agent 1 – Content Analysis Agent

Analyzes text and image model outputs and identifies potentially problematic categories.

Agent 2 – Policy Agent

Maps model categories to the platform's moderation policy and checks whether a predicted category requires an action.

Agent 3 – Context Agent

Considers permitted surrounding information such as conversation context, previous reports or post relationships to reduce decisions based only on isolated words or pixels.

Agent 4 – Risk Assessment Agent

Combines confidence scores and policy evidence into an overall risk level such as Low, Medium, High or Critical.

Agent 5 – Escalation Agent

Routes ambiguous, low-confidence or high-impact decisions to trained human moderators and records the reason for escalation.

Agent Collaboration

Content Analysis → Policy Check → Context Evaluation → Risk Assessment → Decision / Human Escalation

9. Sample Moderation Decision Logic

Risk Score	Risk Level	System Action	Human Review
0.00–0.29	LOW	Allow / continue monitoring	No
0.30–0.59	MEDIUM	Reduce distribution or request review depending on policy	Optional
0.60–0.84	HIGH	Temporary restriction / hold	Yes, depending on policy
0.85–1.00	CRITICAL	Immediate policy action where permitted	Yes / priority review

These thresholds are illustrative design values. A real platform would calibrate thresholds separately for each policy category using validated production data and the cost of false positives and false negatives.

10. Human-in-the-Loop Escalation

Automation should not be treated as infallible. When the system has low confidence, conflicting modality results or a high-impact policy case, the Escalation Agent creates a reviewer task. The human decision is then recorded as feedback.

**AI DECISION → CONFIDENCE CHECK → CLEAR CASE: AUTOMATE |
AMBIGUOUS CASE: HUMAN REVIEW → VERIFIED OUTCOME → MODEL/RULE
FEEDBACK**

11. Analysis and Validation – 10%

A production moderation framework requires continuous performance evaluation. The most important analysis is not simply overall accuracy; it must measure how reliably harmful content is detected and how often legitimate content is incorrectly restricted.

Metric	Purpose in Moderation
Precision	Measures how many flagged cases are actually policy violations.
Recall	Measures how many actual violations are detected.
F1-Score	Balances precision and recall.
False Positive Rate	Measures unnecessary restrictions on legitimate content.
Latency	Measures time from posting to moderation decision.
Throughput	Measures posts processed per second.
Escalation Rate	Measures the proportion sent to human reviewers.
Reviewer Agreement	Measures consistency between human moderation decisions.

12. Sample Validation Results

Test Scenario	Expected Result	Illustrative System Result
Benign text post	Allow	LOW – Allowed
Clearly abusive text	Flag	HIGH – Restricted / Review
Benign image + neutral text	Allow	LOW – Allowed
Image and text give conflicting signals	Review	MEDIUM – Human Review
High-confidence severe policy violation	Immediate action + review	CRITICAL – Action + Priority Review
Previously unseen suspicious pattern	Escalate	MEDIUM/HIGH – Review and investigate

The above results are illustrative validation scenarios for the architecture, not measured performance from a real social-media platform. Actual deployment requires representative, labeled and policy-approved evaluation datasets.

13. Latency-Oriented Processing Design

Because the system must respond within seconds, the processing path should minimize unnecessary data movement and keep frequently used models and reference data close to the processing layer. Lightweight filtering can happen early, while more expensive multimodal analysis is reserved for records that need it.

FAST PATH: INGEST → BASIC FILTER → HIGH-CONFIDENCE MODEL → DECISION

DEEP PATH: INGEST → MULTIMODAL ANALYSIS → AGENT CORRELATION → HUMAN REVIEW

14. Scalability and Reliability

- RDD partitioning distributes large workloads across workers.
- Additional worker nodes can be added as traffic grows.
- Spark fault tolerance allows lost partitions to be recomputed.
- Separate inference services can scale independently for text and image models.
- Caching can reduce repeated computation for frequently reused datasets.
- Queue-based ingestion prevents sudden traffic spikes from overwhelming the processing layer.

15. Innovation & Practical Feasibility

- Multimodal moderation combines text and image evidence instead of treating each modality independently.
- Collaborative agents divide moderation into specialized reasoning tasks.
- Confidence-aware escalation prevents every uncertain case from being automatically removed.
- Human reviewer outcomes can be fed back into model evaluation and controlled rule updates.
- Distributed RDD processing provides a foundation for handling large content volumes.
- Policy-specific thresholds can be tuned independently to balance safety and fairness.

16. Reverse-Engineered End-to-End Example

A user submits a post containing text and an image. The ingestion layer assigns a unique content ID and places the record into the processing stream. Spark distributes the record among its processing partitions. The text path normalizes the caption and produces NLP features. The image path produces visual features. Separate models return confidence scores.

Stage	Output
Ingestion	Content ID = P005

Text analysis	Potential policy-risk score
Image analysis	Visual-risk score
Policy Agent	Applicable policy categories
Context Agent	Supporting/contradicting context
Risk Agent	Combined risk score = illustrative 0.88
Escalation Agent	Priority human review
Decision Engine	Temporary restriction pending review
Human Reviewer	Verified policy outcome
Feedback Store	Decision retained for evaluation/rule improvement

17. Conclusion

The reverse-engineered design demonstrates how a modern social-media moderation platform can process large volumes of user-generated content using distributed computing. PySpark RDD transformations provide a mechanism for partitioning and transforming content records, while specialized text and image analysis models produce evidence for moderation decisions.

Overall, the proposed framework is scalable, multimodal, latency-aware and aligned with a human-in-the-loop moderation approach. It provides a feasible engineering design for automated content moderation without claiming access to any particular platform's proprietary implementation.

18. Future Scope

- Use Spark Structured Streaming for production-grade continuous streams where appropriate.
- Integrate multilingual NLP and multimodal foundation models.
- Use policy-specific calibration and fairness evaluation.
- Add explainable moderation evidence for reviewer interfaces.
- Develop privacy-preserving logging and strict access controls.
- Continuously evaluate models against evolving language, memes and adversarial content.